

RIPHAH INTERNATIONAL UNIVERSITY



Analysis Of Algorithm
Spring 2026

Word Search Puzzle Solver

Supervisor Name: Dr Neelam Umbreen

Project Team

Name of Students	Sap ID	Program	Valid Email Address
Amina Batool	62461	BSSE – 4	62461@students.riphah.edu.pk
Irfa Arshad	63662	BSSE – 4	63662@students.riphah.edu.pk
Nimra Atif	64813	BSSE – 4	64813@students.riphah.edu.pk
Hafsa Saghir	62461	BSSE – 4	65314@students.riphah.edu.pk
Mishal Asim	62516	BSSE – 4	65315@students.riphah.edu.pk
Almarwah K.M. Shloun	71120	BSSE - 4	71120@srudents.riphah.edu.pk

Date of Submission

24/04/2026

Table of Contents

Project Proposal	1
Project Introduction.....	2
Project Title	2
Introduction	2
Problem Statement	2
Proposed Solution	3
Why not other approaches	4
Problem Understanding.....	5
What is the problem.....	6
Input and output.....	6
Real world use cases.....	6
Why is brute force insufficient?	6
Algorithmic Approach	8
Paradigm and Justification	9
High level algorithmic steps.....	9
Key data structures	10
Expected time and space complexity.....	10
Dry Run/Worked Example.....	12
Implementation Plan	20
Techniques used	21
Alternative approach and why not chosen.....	22
Brute force.....	22
KMP	22
Aho Corasick Automation	22
Regex based search	23
Output	23
Limitations of the system	33
Conclusion	34
References	36

Artifact # 1

Project Proposal

Project Introduction

Project Title: Word Search Puzzle Solver using Backtracking, DFS, and Trie-Based Search

Introduction

A word search puzzle is a well-known combinatorial problem in which a set of words are hidden inside a two-dimensional grid of characters. Words may be placed horizontally, vertically, or diagonally, and may run in either forward or reverse direction. The challenge is not merely to find each word by visual inspection, but to design an algorithm that can locate all words efficiently, even in large grids with extensive word lists.

From an algorithmic perspective, this problem is a search problem on a 2D grid with string-matching requirements. It provides a natural playground for exploring the trade-offs between naive brute-force approaches and more sophisticated algorithmic techniques such as backtracking with pruning, depth-first search, and prefix-tree (Trie) based acceleration.

This project aims to implement a correct and efficient Word Search Solver, analyze its time and space complexity, compare it against alternative approaches, and demonstrate its performance through dry runs, test cases, and complexity benchmarks.

This project will develop a fully functional Word Search Puzzle Solver that:

- Accepts any $M \times N$ character grid as input.
- Accepts a dictionary of words to search for.
- Identifies and reports the exact location (row, column, direction) of each found word.
- Demonstrates clear algorithmic thinking through dry runs and complexity analysis.

Problem statement

Given an $M \times N$ grid of characters and a list of K target words, determine whether each word exists in the grid. A word is considered to exist if its characters can be traced in the grid as a contiguous sequence along any of 8 possible directions: right, left, down, up, and the four diagonals. Each cell in the grid may be used at most once per word.

Input	An $M \times N$ 2D character grid and a list of K words to search.
--------------	--

Output	For each word: start cell (row, col), direction (e.g., RIGHT, DOWN-RIGHT), or "NOT FOUND".
Constraint	Grid cells can be visited at most once per word search path. Words may overlap in the grid but not reuse cells within a single word path.
Direction	Right, Left, Down, Up, Down-Right, Down-Left, Up-Right, Up-Left

Proposed solution

Our solution combines two algorithmic techniques to achieve correctness and efficiency:

Primary Technique — Backtracking

Backtracking is a systematic, DFS-based exploration strategy that builds the solution step-by-step and abandons (backtracks) a path as soon as it becomes clear it cannot lead to a valid solution. For the Word Search problem, backtracking works by:

- Starting at each grid cell as a potential word origin.
- Exploring all 8 directions recursively, matching characters one by one.
- Immediately pruning any path where the current prefix does not match the target word.
- Marking cells as visited during exploration and unmarking them when backtracking.

Backtracking is well-suited here because the problem has a well-defined search space (the grid), clear validity checks (character matching, boundary checks), and profitable pruning opportunities (first mismatch terminates that path immediately).

Optimization Technique — Trie (Prefix Tree)

To efficiently handle multiple words simultaneously, we build a Trie (prefix tree) from the dictionary before beginning grid search. The Trie allows us to:

- Check in $O(1)$ per character whether the current path prefix is a valid prefix of any dictionary word.
- Simultaneously search for all K words in a single traversal of the grid.

- Immediately terminate a direction if no word in the dictionary begins with the current prefix.

The search process is implemented using Depth-First Search (DFS) combined with backtracking to systematically explore all valid paths in the grid. This combination reduces redundant exploration dramatically compared to checking each word independently and is the industry-standard technique used in competitive programming for this problem class.

Why not other approaches

Approach	Limitation
Brute Force	Too slow due to repeated full-grid scans
Dynamic Programming	Not suitable due to path-dependent constraints
String Matching (KMP/Regex)	Cannot efficiently handle 2D traversal
Divide & conquer	Breaks word continuity across grid

Artifact # 2

Problem Understanding

Problem Understanding

What is the problem?

A Word Search Puzzle presents the user with a rectangular grid of letters. Hidden within this grid are a number of words — placed horizontally, vertically, or diagonally, in either the forward or reverse direction. The task is to identify exactly where each word starts, in which direction it runs, and to confirm all words are accounted for.

From an algorithmic standpoint, this is a constrained path-finding and string-matching problem on a 2D graph. Each cell is a node, and edges connect it to its 8 neighbors. We seek paths in this graph whose concatenated characters spell a given word.

Input and output?

Input	An $N \times M$ character grid (e.g., 10x10 letters). A list of K words to locate within the grid.
Output	For each word: its starting (row, col) index and direction label (e.g., HORIZONTAL, VERTICAL, DIAGONAL_DOWN_RIGHT, etc.), or the message "NOT FOUND" if the word is absent.

Real world use cases?

Word search solvers are directly used in:

- Educational apps and games that generate and auto-solve puzzles for users.
- DNA sequence searching — scanning a genome string for gene substrings in multiple reading frames.
- Image OCR and document parsing — locating known keyword patterns in a grid of recognized characters.
- Boggle game solvers — finding all valid dictionary words on a letter board (essentially the same algorithm).

Why is brute force insufficient?

A brute-force approach checks every word against every possible starting cell in every direction. For a 15x15 grid, 10 words of avg. length 8:

- Cells: 225, Directions: 8, Length checks: 8, Words: 10 = $225 \times 8 \times 8 \times 10 = 144,000$ -character comparisons.
- For a 100x100 grid with 1,000 words: $100 \times 100 \times 8 \times 10 \times 1,000 = 800$ million comparisons.

The Trie-backed DFS processes all K words in a single traversal. When a path diverges from all Trie prefixes, it backtracks immediately — eliminating the need to restart per-word. This reduces redundant work from $O(K * N * M * 8 * L)$ to effectively $O(N * M * 8 * L)$ in the best case.

Artifact # 3

Algorithmic Approach

Algorithmic Approach

Paradigm and Justification

We apply two complementary paradigms:

Paradigm	Role in this project	Why suitable
Backtracking	Explore grid paths; undo choices when a path cannot spell a target word.	Guarantees all paths are explored; avoids revisiting cells within a word path by maintaining a visited set.
Trie / Prefix Tree	Index all target words to enable $O(1)$ prefix existence checks at each DFS step.	Without Trie, each step would require checking the partial path against all K words linearly. Trie reduces this to a single node lookup.
DFS (Graph Search)	Traverse grid cells from every starting point, following 8 directional edges.	DFS naturally maps to recursive backtracking and is memory-efficient for deep paths.

High level algorithmic steps

1. Insert all target words into a Trie
2. Each word is added letter by letter.
3. The last letter of each word is marked as “word ends here”.
4. Start searching from every cell in the grid
5. If the current cell’s letter matches the first letter of any word in the Trie, begin a path.
 - Explore in all 8 directions
 - Move step by step: right, left, up, down, and all four diagonals.
 - Use DFS + Backtracking to match letters
6. At each step, check the next cell’s letter against the next letter in the Trie.
7. If it matches, move forward and mark the cell as visited (no reuse in same word).

8. If it does not match, stop exploring that direction immediately (pruning). Record a word when fully matched
9. When a Trie node marks the end of a word, store:
 - The word
 - Starting row & column
 - Direction followed
 - Backtrack after exploring each path
 - Unmark the visited cell so other words can reuse it.
 - Continue until all words are found or all cells are explored
 - If a word is never found, report “NOT FOUND”.

Key data structures

Trie (Prefix Tree)	Each node stores a map/array of 26 children (one per letter) and a Boolean/string end-of-word marker. Enables $O(L)$ insertion and $O(1)$ -per-character prefix lookup during DFS.
2D Visited Array	Boolean $N \times M$ grid marking cells already on the current DFS path. Reset (unmark) on backtrack.
Result Map	A dictionary mapping each found word to its starting (row, col) and direction string.
Stack (implicit)	The recursion call stack acts as the DFS stack, tracking the current path depth up to length L .

Expected space and time complexity

Measure	Expression	Explanation
Trie Build Time	$O(K * L)$	K words, avg. length L inserted into Trie.

DFS Search Time	$O(N * M * 8 * L)$	All cells as start; 8 dirs; max depth L. Trie prunes non-matching paths early.
Space Trie	$O(K * L * 26)$	In worst case each character is a unique node.
Space Visited	$O(N * M)$	One Boolean per cell.
Space Stack	$O(L)$	Maximum recursion depth equals longest word.

Artifact # 4

Dry Run/Worked Example

Dry Run/Worked Example

The 5 × 6 Grid

Each cell holds one uppercase letter. Rows are numbered R0–R4 (top to bottom). Columns are numbered C0–C5 (left to right). The three highlighted areas below show where each word is hidden, this is the answer we expect the algorithm to find.

	C0	C1	C2	C3	C4	C5
R0	C	A	T	B	D	F
R1	H	I	J	K	O	L
R2	R	M	P	Q	G	S
R3	V	U	W	X	Y	Z
R4	B	F	N	A	I	J

CAT	(R0, C0) → (R0, C2), Direction: RIGHT	DOG	(R0, C4) → (R2, C4), Direction: DOWN
RUN	(R2, C0) → (R4, C2), Direction: DOWN-RIGHT (Diagonal)		

Words to Search

Sr. No.	Word	length	Expected start	Direction
1	CAT	3	(R0, C0)	RIGHT
2	DOG	3	(R0, C4)	DOWN
3	RUN	3	(R2, C0)	DOWN RIGHT (diagonal)

Searching for CAT

Start: Scan each cell for letter 'C'

The algorithm scans the grid row by row. The first cell is $(R0, C0) = 'C'$. This matches the first letter of CAT.

	C0	C1	C2	C3	C4	C5	
R0	C	A	T	B	D	F	.
R1	H	I	J	K	O	L	.
R2	R	M	P	Q	G	S	.
R3	V	U	W	X	Y	Z	.
R4	B	F	N	H	I	J	.

From $(R0, C0)$, the algorithm tries RIGHT first. It checks $(R0, C1) = 'A'$ — matches the 2nd letter. Then $(R0, C2) = 'T'$ — matches the 3rd letter. All 3 letters matched.

Try all 8 directions from $(R0, C0)$

Attempt	From cell	Letter checked	Grid letter	Match	Action
1	$(R0, C0)$	C	C	Yes	Continue RIGHT
2	$(R0, C1)$	A	A	Yes	Continue RIGHT
3	$(R0, C2)$	T	T	Yes	All letters matched.

Result for CAT

	C0	C1	C2	C3	C4	C5	
R0	C	A	T	B	D	F	
R1	H	I	J	K	O	L	
R2	R	M	P	Q	G	S	
R3	V	U	W	X	Y	Z	
R4	B	F	N	H	I	J	

word = CAT
 Status = Found
 Start cell = (Row 0, Column 0)
 End cell = (Row 0, Column 2)
 Direction = RIGHT
 Path = C(R0,C0) → A(R0,C1) → T(R0,C2)

Searching for DOG

Scan for letter 'D'

The algorithm scans all cells. It reaches (R0, C4) = 'D'. This matches the first letter of DOG.

	C0	C1	C2	C3	C4	C5	
R0	C	A	T	B	D	F	
R1	H	I	J	K	O	L	
R2	R	M	P	Q	G	S	
R3	V	U	W	X	Y	Z	
R4	B	F	N	H	I	J	

Try directions from (R0, C4)

First, the algorithm tries RIGHT: (R0, C5) = 'F', does NOT match 'O'. Prune. Then tries DOWN: (R1, C4) = 'O', matches! Then (R2, C4) = 'G', matches! All 3 letters found.

Attempt	Direction	Cell	Letter checked	Grid letter	Match	Action

1	RIGHT	(R0, C5)	O	F	No	PRUNE, try next direction
2	DOWN	(R1, C4)	O	O	Yes	Continue DOWN
3	DOWN	(R2, C4)	G	G	Yes	All letters matched

When the algorithm tried RIGHT from (R0, C4), it checked (R0, C5) = 'F' which is NOT 'O'. It immediately stopped and did not check any further in that direction. This is pruning, saving unnecessary work.

Result for DOG

	C0	C1	C2	C3	C4	C5
R0	C	A	T	B	D	F
R1	H	I	J	K	O	L
R2	R	M	P	Q	G	S
R3	V	U	W	X	Y	Z
R4	B	F	N	H	I	J

word = DOG
 Status = Found.
 start cell = (Row 0, Column 4)
 end cell = (Row 2, Column 4)
 directions = DOWN
 Path = D(R0,C4) → O(R1,C4) → G(R2,C4)

Searching for RUN

Scan for letter 'R'

The algorithm scans the grid and reaches (R2, C0) = 'R'. This matches the first letter of RUN.

	C0	C1	C2	C3	C4	C5
R0	C	A	T	B	D	F
R1	H	I	J	K	O	L
R2	R	M	P	Q	S	S
R3	V	U	W	X	Y	Z
R4	B	F	N	H	I	J

Try all directions from (R2, C0)

The algorithm tries directions one by one. RIGHT gives (R2, C1) = 'M', not 'U', prune. DOWN gives (R3, C0) = 'V' — not 'U', prune. Then DOWN-RIGHT: (R3, C1) = 'U', matches! Then (R4, C2) = 'N' — matches! Word found diagonally.

Attempts	Direction	Cell	Letter checked	Grid letter	Match	Action
1	RIGHT	(R2, C1)	U	M	No	PRUNE
2	LEFT	(R2, C-1)	-	-	No	Out of bounds, SKIP
3	DOWN	(R3, C0)	U	V	No	PRUNE
4	UP	(R1, C0)	U	H	No	PRUNE
5	DOWN RIGHT	(R3, C1)	U	U	Yes	Continue DOWN RIGHT
6	DOWN RIGHT	(R4, C2)	N	N	Yes	All letters matched

Diagonal path visualized

The DOWN-RIGHT direction means: row increases by 1, column increases by 1 at each step. So, the path goes diagonally from top-left to bottom-right.

	C0	C1	C2	C3	C4	C5
R0	C	A	T	B	D	F
R1	H	I	J	K	O	L
R2	R	M	P	Q	G	S
R3	V	U	W	X	Y	Z
R4	B	F	N	H	I	J

word = RUN
 Status = Found
 Start cell = (Row 2, Column 0)
 end cell = (Row 4, Column 2)
 direction = DOWN RIGHT (Diagonal)
 Path = R(R2,C0) → U(R3,C1) → N(R4,C2).

Final result summary

After searching all cells in all directions for all three words, the algorithm produces the following results:

	C0	C1	C2	C3	C4	C5
R0	C	A	T	B	D	F
R1	H	I	J	K	O	L
R2	R	M	P	Q	G	S
R3	V	U	W	X	Y	Z
R4	B	F	N	H	I	J

Word	Status	Start	End	Direction	Path
CAT	Found	(R0, C0)	(R0, C2)	RIGHT	C – A – T
DOG	Found	((R2, C4)	(R2, C4)	DOWN	D – O – G

RUN	Found	(R2, C0)	(R4, C2)	DOWN RIGHT	R – U – N
-----	-------	----------	----------	---------------	-----------

All 3 words were successfully found. The algorithm correctly identified words in three different directions — RIGHT, DOWN, and DOWN-RIGHT (diagonal) — demonstrating that the backtracking approach works for all supported directions.

Artifact # 5

Implementation Plan

Implementation plan

Language	C++ for its built-in STL data structures (unordered map, vector, string), efficient recursion, and struct-based Trie nodes. Python is available as a secondary implementation for rapid prototyping.
Output Method	The program takes input directly from the user via console (keyboard input), including the grid dimensions, grid characters, and the list of words.
Output Verification	Results are printed to console and also written to an output file: each word followed by its start coordinates and direction, or NOT FOUND. Automated test cases compare output against a known-correct answer file.
Anticipated Challenges	1. Correctly implementing Trie deletion after a word is found (to avoid re-reporting). 2. Handling diagonal boundary conditions. 3. Ensuring visited [] [] is properly reset during backtracking. 4. Performance on very large grids (1000x1000), may require iterative DFS to avoid stack overflow.

Techniques Used

Technique	How it is applied	Why chosen
Trie (Prefix Tree)	All K words indexed before search. DFS queries Trie at each step.	Simultaneous multi-word search. Prunes paths with no matching prefix in O (1) per step.

Backtracking DFS	Recursive exploration of grid from each starting cell. Visited array prevents cell reuse.	Systematic, complete exploration. Backtracking ensures no path is missed while avoiding repeated work.
8-Directional Traversal	At each DFS step, all 8 neighbors are evaluated as possible next cells.	Word search grids allow words in all 8 directions, must be covered for correctness.
Visited Marking	A boolean $N \times M$ array marks cells on the current DFS path.	Prevents a single cell from being used twice in the same word path (standard word search rule).

Alternative Approaches and Why Not Chosen

Brute Force (Naive String Match)

For each word W , scan every cell (i,j) in every direction and compare characters one by one. Time: $O(N \times M \times 8 \times L \times K)$. Simple to implement but completely redundant, every word restarts the full grid scan. Rejected because the Trie eliminates this redundancy by searching all K words simultaneously.

KMP (Knuth-Morris-Pratt) per row/column/diagonal

Apply KMP on every row, column, and diagonal of the grid for each word. KMP runs in $O(N+L)$ per line. For all lines and all words: $O(K \times (N + M + \text{diagonals}) \times \text{max_dim})$. This works well for purely horizontal/vertical puzzles but handling diagonals adds complexity. It also does not naturally extend to 2D patterns. Rejected because DFS + Trie handles all 8 directions more uniformly and scales better when K is large.

Aho-Corasick Automaton

Aho-Corasick builds a failure-link automaton over all K words and runs a single linear scan of each row/column/diagonal in $O(N \times M + \text{total matches})$. It is theoretically the most efficient for large K . However, it requires building failure links (non-trivial implementation) and does not naturally handle 2D diagonal paths. We chose Trie + DFS because the implementation complexity is more appropriate for this course, while still being asymptotically efficient. Aho-Corasick is noted as a scope extension.

Regex-Based Search

Apply regular expressions on each row, column, and diagonal string. Very high-level and easy to write, but entirely opaque in terms of algorithmic complexity. Provides no insight into data structures or search paradigms, unsuitable for an algorithms course project.

Approach	Time complexity	Handles 8 directions	Chosen
Brute force	$O(N * M * 8 * L * K)$	Yes	No, too slow for large K
KMP per line	$O(K * (N + M) * \max)$	Limited	No, poor diagonal support
Aho corasick	$O(N * M + \text{matches})$	Limited	No, complex
Trie + DFS	$O(N * M * 8 * L)$	Yes	Yes, correct, efficient, clear
Regex	Unknown / opaque	Limited	No, not algorithmically meaningful for this problem

Output

```

=====
WORD SEARCH PUZZLE SOLVER
Trie + DFS Backtracking Algorithm
Design & Analysis of Algorithms
Instructor: Dr. Neelam Umbreen
=====

=====
MAIN MENU
=====
1 -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2 -> Enter your own puzzle
3 -> Exit

Your choice:

```

```

=====
WORD SEARCH PUZZLE SOLVER
Trie + DFS Backtracking Algorithm
Design & Analysis of Algorithms
Instructor: Dr. Neelam Umbreen
=====

=====
MAIN MENU
=====
1 -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2 -> Enter your own puzzle
3 -> Exit

Your choice: 1|

```

```
=====
WORD SEARCH PUZZLE SOLVER
Trie + DFS Backtracking Algorithm
Design & Analysis of Algorithms
Instructor: Dr. Neelam Umbreen
=====

=====
MAIN MENU
=====
1 -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2 -> Enter your own puzzle
3 -> Exit

Your choice: 1
[Dry-run: 5x5 grid  5 · RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT]

=====
WORDS TO SEARCH
=====
1. ALGO
2. RICE
3. OAK
4. SEA

=====
PUZZLE GRID
```

```
=====
                        PUZZLE GRID
=====
      C0  C1  C2  C3  C4
    +---+---+---+---+
R0  | A | L | G | O | R |
    +---+---+---+---+
R1  | F | O | B | N | I |
    +---+---+---+---+
R2  | M | M | A | G | C |
    +---+---+---+---+
R3  | F | E | Z | K | E |
    +---+---+---+---+
R4  | S | D | V | C | B |
    +---+---+---+---+

=====
STEP 1: Building Trie...
=====
Inserted "ALGO"  -> A -> L -> G -> O [END]
Inserted "RICE"  -> R -> I -> C -> E [END]
Inserted "OAK"   -> O -> A -> K [END]
Inserted "SEA"   -> S -> E -> A [END]
Done. Total Trie nodes: 15

=====
STEP 2: DFS Backtracking Search...
=====
Search complete!

=====
                        SEARCH RESULTS
=====
Word      Status      Start      End      Direction
-----
ALGO      FOUND      (0,0)      (0,3)      RIGHT
RICE      FOUND      (0,4)      (3,4)      DOWN
OAK       FOUND      (1,1)      (3,3)      DOWN-RIGHT
SEA       FOUND      (4,0)      (2,2)      UP-RIGHT
=====
```

Found: 4 Not Found: 0 Total: 4

COLOR-HIGHLIGHTED GRID

Legend: ALGO RICE OAK SEA

	C0	C1	C2	C3	C4
R0	A	L	G	O	R
R1	F	O	B	N	I
R2	M	M	A	G	C
R3	F	E	Z	K	E
R4	S	D	V	C	B

COMPLEXITY ANALYSIS

Grid Size : 5 x 5
Words (K) : 4
Max Length (L): 4 characters

BRUTE FORCE - searches each word separately:
Formula : $O(N \times M \times 8 \times L \times K)$
= $5 \times 5 \times 8 \times 4 \times 4$ = 3200 operations

TRIE + BACKTRACKING - all words in one pass:
Formula : $O(N \times M \times 8 \times L)$
= $5 \times 5 \times 8 \times 4$ = 800 operations

Speedup: 4x faster than brute force

SPACE COMPLEXITY:
Trie : $O(K \times L \times 26)$ ~ 416 pointers
Visited : $O(N \times M)$ = 25 booleans
Stack : $O(L)$ = 4 recursive frames

Press ENTER to return to menu...|

```
=====
WORD SEARCH PUZZLE SOLVER
Trie + DFS Backtracking Algorithm
Design & Analysis of Algorithms
Instructor: Dr. Neelam Umbreen
=====

=====
MAIN MENU
=====
1 -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2 -> Enter your own puzzle
3 -> Exit

Your choice: 2
```

```

=====
WORD SEARCH PUZZLE SOLVER
Trie + DFS Backtracking Algorithm
Design & Analysis of Algorithms
Instructor: Dr. Neelam Umbreen
=====

=====
MAIN MENU
=====
1 -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2 -> Enter your own puzzle
3 -> Exit

Your choice: 2

Enter grid dimensions:
Rows (max 20): 5
Cols (max 20): 6

Enter grid row by row (no spaces):
Row 0: Struvy
Row 1: Polmeo
Row 2: Bpiahu
Row 3: Ruveme
Row 4: Sdeuha

How many words to search? 6
Enter the words:
Word 1: Top
Word 2: Lama
Word 3: Run
Word 4: You
Word 5: Sport
Word 6: Ruve

```

WORDS TO SEARCH

1. TOP
2. LAMA
3. RUN
4. YOU
5. SPORT
6. RUVE

PUZZLE GRID

	C0	C1	C2	C3	C4	C5
R0	S	T	R	U	V	Y
R1	P	O	L	M	E	O
R2	B	P	I	A	H	U
R3	R	U	V	E	M	E
R4	S	D	E	U	H	A

STEP 1: Building Trie...

```
Inserted "TOP"   -> T -> O -> P [END]
Inserted "LAMA"  -> L -> A -> M -> A [END]
Inserted "RUN"   -> R -> U -> N [END]
Inserted "YOU"   -> Y -> O -> U [END]
Inserted "SPORT" -> S -> P -> O -> R -> T [END]
Inserted "RUVE"  -> R -> U -> V -> E [END]
Done. Total Trie nodes: 21
```



```
-----
STEP 2: DFS Backtracking Search...
-----
Search complete!

=====
                        SEARCH RESULTS
=====
Word      Status      Start      End      Direction
-----
TOP       FOUND       (0,1)     (2,1)     DOWN
LAMA      FOUND       (1,2)     (4,5)     DOWN-RIGHT
RUN       NOT FOUND
YOU       FOUND       (0,5)     (2,5)     DOWN
SPORT    NOT FOUND
RUVE      FOUND       (3,0)     (3,3)     RIGHT
=====
Found: 4   Not Found: 2   Total: 6

=====
                        COLOR-HIGHLIGHTED GRID
=====
Legend:  TOP  LAMA  YOU  RUVE

      C0  C1  C2  C3  C4  C5
+---+---+---+---+---+
R0 | S | T | R | U | V | Y |
+---+---+---+---+---+
R1 | P | O | L | M | E | O |
+---+---+---+---+---+
R2 | B | P | I | A | H | U |
+---+---+---+---+---+
R3 | R | U | V | E | M | E |
+---+---+---+---+---+
R4 | S | D | E | U | H | A |
+---+---+---+---+---+
```

```

=====
                        COMPLEXITY ANALYSIS
=====
Grid Size      : 5 x 6
Words (K)      : 6
Max Length (L): 5 characters

BRUTE FORCE - searches each word separately:
Formula :  $O(N \times M \times 8 \times L \times K)$ 
= 5 x 6 x 8 x 5 x 6 = 7200 operations

TRIE + BACKTRACKING - all words in one pass:
Formula :  $O(N \times M \times 8 \times L)$ 
= 5 x 6 x 8 x 5 = 1200 operations

Speedup: 6x faster than brute force

SPACE COMPLEXITY:
Trie      :  $O(K \times L \times 26)$  ~= 780 pointers
Visited   :  $O(N \times M)$       = 30 booleans
Stack     :  $O(L)$               = 5 recursive frames
=====

```

```

=====
                        WORD SEARCH PUZZLE SOLVER
                        Trie + DFS Backtracking Algorithm
                        Design & Analysis of Algorithms
                        Instructor: Dr. Neelam Umbreen
=====

=====
                        MAIN MENU
=====
1  -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2  -> Enter your own puzzle
3  -> Exit

Your choice: 3

```

```
=====
WORD SEARCH PUZZLE SOLVER
Trie + DFS Backtracking Algorithm
Design & Analysis of Algorithms
Instructor: Dr. Neelam Umbreen
=====

=====
MAIN MENU
=====
1 -> Dry-run example (5x5 grid: RIGHT, DOWN, DOWN-RIGHT, UP-RIGHT)
2 -> Enter your own puzzle
3 -> Exit

Your choice: 3
```

```
=====
Thank you! Goodbye.
=====

-----
Process exited after 497.9 seconds with return value 0
Press any key to continue . . .
```

Limitations of the System

Despite achieving correct and efficient word search functionality, the current implementation has the following limitations:

1. Grid Size Limitation

The system is designed to handle grids up to a fixed size (e.g., 20×20). Larger grids may require dynamic memory allocation and further optimization.

2. Single Occurrence Reporting

If a word appears multiple times in the grid, the system reports only the first occurrence found and ignores subsequent matches.

3. Memory Management

The Trie data structure uses dynamic memory allocation but does not explicitly free memory after execution. While acceptable for short-lived programs, this may not be optimal for large-scale systems.

4. Character Set Restriction

The implementation assumes input consists of uppercase English alphabets (A–Z). Other character sets are not supported.

5. Recursion Depth Limitation

The use of recursive DFS may lead to stack overflow for extremely large grids or very long words.

6. No Parallel Processing

The algorithm runs sequentially and does not utilize multi-threading or parallel processing, which could improve performance on large datasets.

Conclusion

This project successfully presents the design and implementation of a Word Search Puzzle Solver using a combination of Depth-First Search (DFS), backtracking, and Trie-based optimization. The solution efficiently locates words within a two-dimensional grid by exploring all valid paths while eliminating unnecessary computations through prefix pruning.

The use of a Trie data structure significantly improves performance by enabling simultaneous multi-word searching and reducing redundant checks. The DFS with backtracking approach ensures complete exploration of the search space while maintaining correctness by preventing cell reuse within a single word path. Together, these techniques provide a balanced solution that is both efficient and conceptually clear.

Through complexity analysis and practical implementation, the project demonstrates how algorithmic design choices impact performance, especially when scaling to larger grids and word sets. The comparison with alternative approaches highlights the advantages of the chosen method in terms of efficiency, flexibility, and suitability for two-dimensional search problems.

Although the current implementation has certain limitations, such as fixed grid size and single-occurrence reporting, it provides a strong foundation for further enhancements. Future improvements may include support for larger dynamic inputs, detection of multiple occurrences of words, and optimization using parallel processing techniques.

Overall, this project effectively illustrates the application of fundamental algorithmic paradigms and data structures to solve a real-world problem, reinforcing key concepts in the design and analysis of algorithms.

Artifact # 6

References

References

The following resources were consulted during the preparation of this proposal:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. [Chapter 26 — Graph algorithms; Chapter 12 — Binary Search Trees for Trie analogy]
2. Sedgewick, R. & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. [Chapter 5 — Strings: Tries and substring search]
3. Levitin, A. (2012). Introduction to the Design and Analysis of Algorithms (3rd ed.). Pearson. [Chapter 7 — Space and Time Trade-offs]
4. LeetCode Problem #212 — Word Search II (Trie + DFS formulation reference for understanding, not copied). Available: <https://leetcode.com/problems/word-search-ii/>